# How Is AI Actually Producing New Mathematics?

A discussions of verifiers and pushing reasoning

Patrick Ming

September 14th, 2026

Core papers: arXiv:2605.22763 · arXiv:2601.22401 · arXiv:2506.13131 · Tzachristas et al., ACL 2026



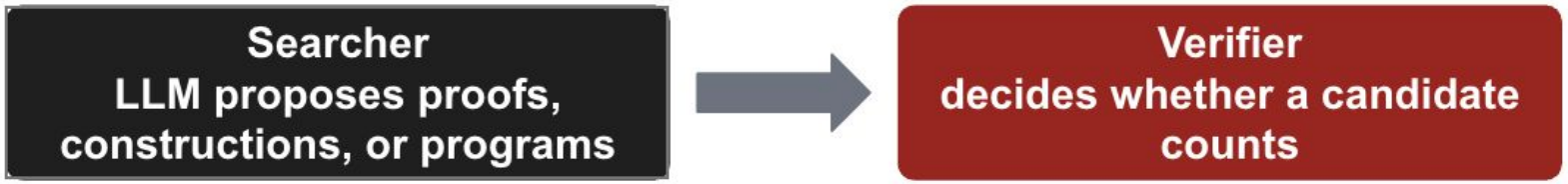
# Recent Developments in Math aided by AI

- OpenAI and Google solving Erdös problems
- Fable finding a counterexample to Jacobian Conjectures
- Claude pushing lower bound of zeros satisfying Riemann function from 41.6% to 67.2%
- OpenAI (allegedly) solving Navier Stokes

Open questions:

- What defines a meaningful contribution?
- What are current methods of pushing reasoning/solving problems?
  - What is the verifier?

# Search + Verifier Architecture




Examples of such verifiers: (and where we will focus our efforts)

- Formal verifiers (Lean)
- Natural Language based discovery (humans and literature)
- Program search (unit tests, cheap, good for constructions)

Verifiers drive results, but not reasoning, how can we build on the latter?

# Four Tiers of Discovery

| | | |
|---|---|---|
| A | **Tier A · Formal** | A formal proof checked by a proof assistant (Lean/Coq/Isabelle), or a proof-carrying artifact where verification is fully automated. |
| B | **Tier B · Certified** | A machine-checkable certificate: explicit construction plus deterministic checker, inequality certificate, or reproducible code proving a bound. |
| C | **Tier C · Reproducible best-known** | A new best-known result backed by an open evaluator, strong baselines, and ablations; verification may need heavy computation but is independently runnable. |
| D | **Tier D · Suggestive** | Plausible conjectures, heuristics, or partial progress without a verifiable certificate. Valuable, but not "solved." |

**Survey conclusion: "the strongest results are those where verification is cheap relative to generation."**

Tzachristas, Tzachristas & Sui, "Open Problems Solved by LLMs? A Survey of Verifiable Mathematical Discovery," ACL 2026 Big Picture workshop, pp. 10–21

# General Outline for Presentation

1

## When the verifier is a type-checker (formal)

Paper: AI-driven formal proof search on Erdős and OEIS problems

2

## When the verifier is a person (natural-language driven)

Paper: Gemini Deep Think on 700 “open” Erdős problems

3

## When the verifier is a score (program)

AlphaEvolve and program search

4

## Discussion on Reasoning Approaches

Paper: Olympiad-level Formal Mathematical Reasoning with Reinforcement Learning

*Discussion pauses after Parts 1 and 2 (a few minutes each).*



PART 1

# When the verifier is a type-checker

Advancing Mathematics Research with AI-Driven Formal Proof Search · arXiv:2605.22763

# Paper Overview

- **Input: a Lean file with the formal statement and an empty proof body.**
- Prover subagents run multi-turn loops: reason in chain-of-thought, edit the proof sketch with search-and-replace tools, compile, read the error, repeat.
- A proof counts when Lean reports no remaining goals and no sorry. Nothing else counts.
- Prover model: Gemini 3.1 Pro. Rating model for the evolutionary variant: Gemini 3.0 Flash. Smaller models (3.0 Flash, 3.1 Flash-Lite) solved nothing.
- Erdős formal statements: 353 at the time of the run. OEIS: 492 open questions auto formalized by Gemini, then attacked.

## Four agent variants (the ablation axis)

**1** **Basic** independent prover subagents; generate, compile, retry. No shared state.

**2** **+ AlphaProof** queries the RL prover to fill proof gaps. AlphaProof returns proof/disproof/failure

**3** **Evolutionary** shared population of sketches, Elo-rated, sampled with P-UCB to start next episode

**4** **Full** evolutionary search plus AlphaProof.

Tsoukalas, Kovsharov, Shirobokov et al. (Google DeepMind), “Advancing Mathematics Research with AI-Driven Formal Proof Search,” arXiv:2605.22763, May 2026 · see paper Fig. 1 (example input/output on Erdős #125) and Fig. 2 (full agent design)

# General Results on Erdős problems and OEIS Conjectures

**9 / 353**

open Erdős problems resolved autonomously (formal statements at time of run, 4 genuinely novel results)

**44 / 492**

OEIS conjectures proved, after Gemini autoformalized the 492 statements

**~$100s**

reported per-problem inference cost (“a few hundred dollars”)

**27.5 TPU-h**

AlphaProof compute per problem (≈ $60 on v6e)

- Examples the paper highlights: #12(i) and #12(ii), posed by Erdős and Sárközy in 1970; #125, open since 1996, needed an “inductive thinning” argument exploiting Diophantine structure.
- All nine are Tier A on the evidence ladder: the artifact is a Lean proof anyone can re-check.
- Successes cluster in combinatorics, convex optimization, and number theory, i.e. where mathematical library is mature and problems decompose into tractable subgoals.

Tsoukalas, Kovsharov, Shirobokov et al. (Google DeepMind), “Advancing Mathematics Research with AI-Driven Formal Proof Search,” arXiv:2605.22763, May 2026

# How Which Approach Changes Cost

| Comparison | What the paper reports (Fig. 3: solve rate vs. mean inference cost, six solved Erdős problems) |
|---|---|
| A (basic) vs. B (+AlphaProof) | Within the margin of error on 4 of the 6 problems. |
| D (full) on #125 and #138 | Outperforms A and B, with 2× to 5× monetary savings on the hardest instances. |
| D (full) on the other four | Roughly half as cost-efficient as A and B. |
| Smaller models in any variant | Gemini 3.0 Flash and 3.1 Flash-Lite solved none of the problems. |
| Bottom line (authors' wording) | "A basic agent alternating LLM-based generation with Lean-based verification replicated the Erdős successes but proved costlier on the hardest problems." |

## My Interpretation

Correctness comes from the verifier: the dumbest loop finds the same proofs.

Search structure (population, Elo, AlphaProof calls) changes the cost curve, and only on the hardest instances.

Model capability is a threshold: nothing below Gemini 3.1 Pro.

Tsoukalas, Kovsharov, Shirobokov et al. (Google DeepMind), "Advancing Mathematics Research with AI-Driven Formal Proof Search," arXiv:2605.22763, May 2026 · paste Fig. 3 here if you have time

# Limitations

## The formalization gap

A mis-stated theorem still compiles. The authors found and fixed misformalizations in their own inputs: in #125 and #741(i), “density” had to be amended to “lower density” and “upper density” after the agent found proofs.

## Hallucinated lemmas

Top sketches sometimes leaned on lemmas left as sorry that the agent claimed were established results in the literature. On manual inspection, “these lemmas proved to be hallucinations.” Lean catches this only because sorry is not a proof.

## Coverage and insight

Wins concentrate where the math library is mature. “Even most Erdős problems remain out of reach, let alone problems that require extensive new theory.” And a compiled proof can be unreadable; correctness is not understanding.

Tsoukalas, Kovsharov, Shirobokov et al. (Google DeepMind), “Advancing Mathematics Research with AI-Driven Formal Proof Search,” arXiv:2605.22763, May 2026

# Limitations

- Formalization Gap:
  - Authors had to rewrite problem statements after finding misformalizations in #125, #741(i)
- Hallucinations: (Were caught by Lean)
  - Agents sometimes hallucinated lemmas. Logical gap then cascades through the proof
- Coverage:
  - Success concentrated around fields with a well-established math library. Sometimes correct proofs were also unreadable.

Tsoukalas, Kovsharov, Shirobokov et al. (Google DeepMind), “Advancing Mathematics Research with AI-Driven Formal Proof Search,” arXiv:2605.22763, May 2026

PART 2

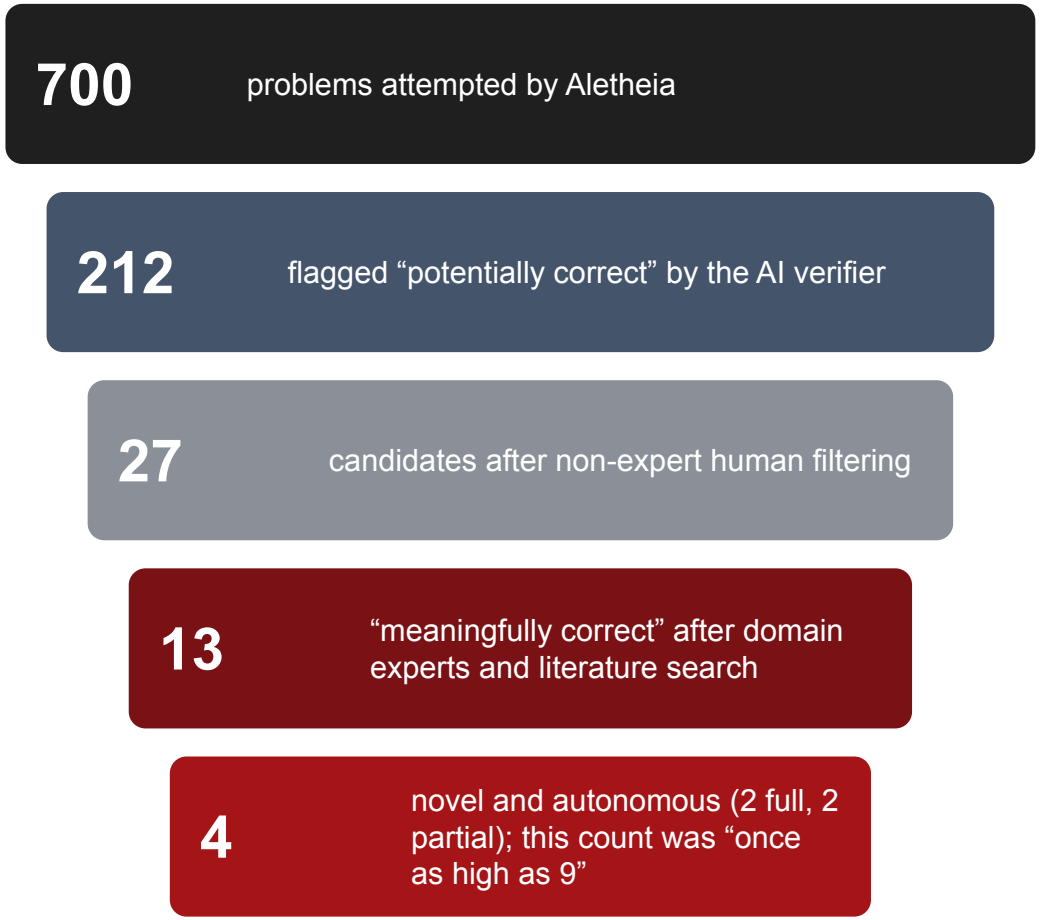
# When the verifier is a person (control exp.)

Gemini Deep Think on 700 “open” Erdős problems · arXiv:2601.22401

# Alethia; AI and Human Verifiers

PART 2 · SETUP

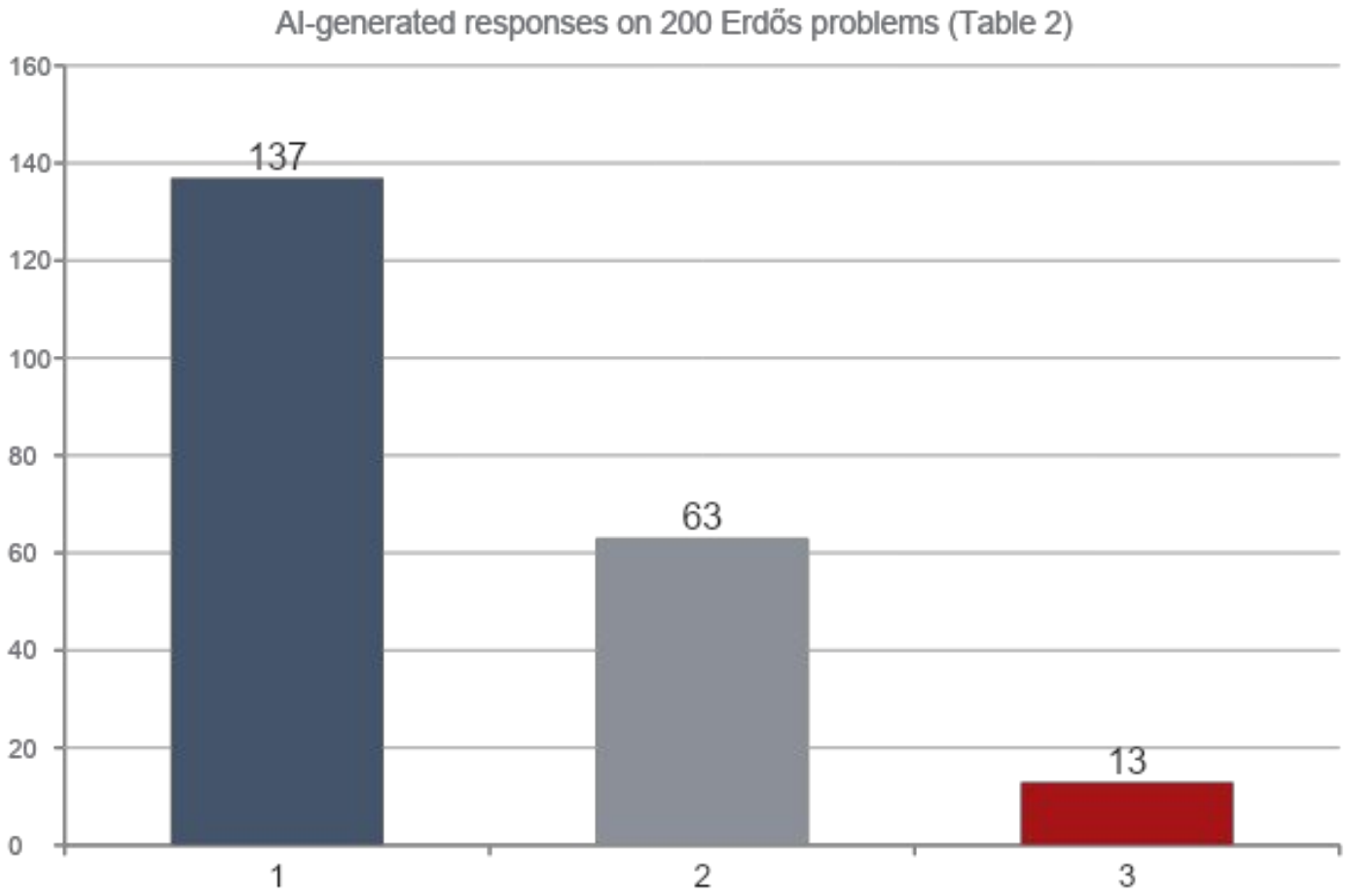
- **A week spent trying to solve 700 conjectures labeled open in Bloom's Erdős Problems database.**
- Aletheia is a research agent wrapping Gemini Deep Think. It includes a natural-language verifier that scores its own candidate solutions.
- No Lean in the loop. The paper discusses formal verification as an alternative and notes it is limited by how little of the literature is formalized.
- Authors' stance: "mathematics papers should always be authored by humans, even when the AI-generated content is (fully) mathematically correct."




Feng, Trinh, Bingham et al. (Google DeepMind et al.), "Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems," arXiv:2601.22401v3, Aug 2026 · funnel numbers from Sections 1–2 and Tables 1–2

# Results

PART 2 · THE SIGNAL-TO-NOISE RATIO




## Two different kinds of “correct”

Technically correct (31.5%): the argument is valid for the statement as the model read it (but strayed from the intended problem).

Meaningfully correct (6.5%): valid for the problem Erdős actually intended.

The gap is definitional drift: Aletheia was not told Bloom’s conventions and “would commonly confuse different (valid) interpretations of technical terms (e.g., additive versus Dirichlet convolution, strong versus weak completeness).”

Feng, Trinh, Bingham et al. (Google DeepMind et al.), “Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems,” arXiv:2601.22401v3, Aug 2026 · Table 2

# How Were These Problems Solved?

| | | |
|---|---|---|
| 2 | **Autonomous resolution** | Erdős 652, 1051 |
| 2 | **Partial AI solution** | Erdős 654, 1040 |
| 4 | **Independent rediscovery** | 397, 659, 935, 1089: correct, but a solution already existed in the literature |
| 5 | **Literature identification** | 333, 591, 705, 992, 1105: the model found the existing solution rather than producing one |

### The authors on the bottleneck

“Perhaps surprisingly, the lengthiest and most arduous step of our effort was the final one of investigating whether the solutions were already in the literature, and whether they really addressed the intended problem.”

And on the four novel ones: “none of the four individually rises to the level of a research paper.”

**Here, the verifier becomes checking the library, which is expensive.**

Feng, Trinh, Bingham et al. (Google DeepMind et al.), “Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems,” arXiv:2601.22401v3, Aug 2026 · Table 1 (taxonomy of Aletheia results)

# “Failure” Mode: Rediscovery

PART 2 · THREE REDISCOVERIES

INDEPENDENT REDISCOVERY

## Erdős 1089

Answered by “an offhand remark in a 1981 paper, where the authors seemed unaware that they had resolved an Erdős problem.”

The result existed for 44 years without anyone connecting it to the problem list.

INDEPENDENT REDISCOVERY

## Erdős 397

Later identified as “a variant of Problem 3 from Day 1 of the 2012 Chinese Team Selection Test for the IMO.”

GPT-5.2 Pro and Harmonic’s Aristotle independently found the same counterexample.

LITERATURE IDENTIFICATION

## Erdős 333

A claimed human-plus-AI solution turned out to have been published by Erdős himself in 1977. The author publicly asked the community “to put greater focus on literature search on the problems currently marked as open.”

*The authors’ broader suspicion: for all AI solutions not yet located in the literature, “we find it plausible that they were also discovered before by humans years ago … but were never published because they were not considered important enough.”*

Feng, Trinh, Bingham et al. (Google DeepMind et al.), “Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems,” arXiv:2601.22401v3, Aug 2026 · Quanta Magazine, “Why the Legendary Erdős Problems Are Falling to AI,” Aug 3 2026 (Problem 333 account)

# Small Summary

PARTS 1 AND 2 SIDE BY SIDE

| | Formal proof search (arXiv:2605.22763) | Natural-language discovery (arXiv:2601.22401) |
|---|---|---|
| **Verifier** | Lean type-checker | AI verifier → non-experts → domain experts → literature |
| **Soundness** | Exact for the stated theorem | “Slightly subjective, even when performed by human experts” (5 mathematicians disagreed on 1051) |
| **Headline yield** | 9 / 353 Erdős, 44 / 492 OEIS | 700 → 13 meaningfully correct → 4 novel (2 full, 2 partial) |
| **Cost** | “A few hundred dollars” per problem, reported | Not reported; the expensive part was human literature search |
| **Dominant failure** | Formalization gap; hallucinated sorry lemmas (caught) | Definitional drift; rediscovery; possible indirect ingestion (not caught automatically) |
| **Evidence tier** | A | D until a human promotes it; the paper’s 13 are human-verified |
| **What it produces** | A compiled proof, possibly unreadable | A readable argument, possibly already published |

Tsoukalas, Kovsharov, Shirobokov et al. (Google DeepMind), “Advancing Mathematics Research with AI-Driven Formal Proof Search,” arXiv:2605.22763, May 2026 · Feng, Trinh, Bingham et al. (Google DeepMind et al.), “Semi-Autonomous Mathematics Discovery with Gemini: A Case Study on the Erdős Problems,” arXiv:2601.22401v3, Aug 2026



PART 3

# When the verifier is a score

AlphaEvolve and program search · arXiv:2506.13131

# Evolve programs, score them, keep the best

**LLM ensemble**
proposes code edits to the current program

→

**Program database**
stores candidates and their scores; samples parents

→

**Evaluator**
runs the program, returns a numeric score

→

**Selection**
keeps improving candidates; loop repeats

↺ *the evaluator's score is the only feedback; no proof, no human, in the inner loop*

**48**

scalar multiplications for 4×4 complex matrices: "the first improvement, after 56 years, over Strassen's algorithm"

## Takeaway: Verifier is cheap but scope is limited

The verifier is a function call. For a matrix-multiplication scheme the checker is exact. For a kernel speedup it is a benchmark.

The limit: it only applies when the object is constructive and "better" is computable. No Lean, no readable proof, and nothing for problems whose answer is a theorem rather than an object.

Novikov et al. (Google DeepMind), "AlphaEvolve: A coding agent for scientific and algorithmic discovery," arXiv:2506.13131, Jun 2025 · the 48-multiplication result and the loop description are from the paper's abstract and Section 2

# Three verifiers, three failure modes

SYNTHESIS

PART 1 · TIER A

## Type-checker

Exact on the stated theorem.

Fails at the boundary: is the statement the problem? Is the lemma real or a sorry? Is Mathlib deep enough here?

PART 2 · TIER D → HUMAN-PROMOTED

## Human + literature

Produces what mathematicians actually want: a readable argument.

Fails silently: definitional drift, rediscovery, possible ingestion from training data. The expensive step is the library, not the proof.

The only verifier that can judge significance.

PART 3 · TIER B OR C

## Numeric score

Cheapest of the three; the object is its own certificate.

Fails when the score is a proxy: benchmarks get gamed under search pressure.

Only reaches constructive problems.

**Common thread: in all 3 methods, the model sometimes will solve slightly different problems than the one asked because of hallucinations.**

Tzachristas, Tzachristas & Sui, “Open Problems Solved by LLMs? A Survey of Verifiable Mathematical Discovery,” ACL 2026 Big Picture workshop, pp. 10–21 (tiers) · group AI4AI talk, Aug 31 2026 (evaluator exploits)



PART 4

# Outside of Discovery, What About Reasoning?

RL in math: AlphaProof · Hubert et al., 2025

# Mathematical Reasoning Model: AlphaProof

**State**
the current Lean tactic state: hypotheses and open goals

→

**Proof network**
3B encoder–decoder; outputs N candidate tactics and a value (difficulty)

→

**Tree search**
AlphaZero-style; expands nodes with the network's tactics and values

→

**Lean**
applies the tactic; returns new state, or proof / disproof / timeout

*Reward: −1 for every tactic applied, so a found proof is rewarded for being short; the episode ends only on a Lean-verified proof, disproof, or timeout. Nothing else is ever rewarded.*

**~1M → ~80M**

informal problems autoformalized into Lean statements by a fine-tuned Gemini model (Fig. 2a)

**~80,000**

TPU-days of main RL training on that curriculum

**300B / 300k**

pretraining tokens (code + math) / Mathlib state–tactic pairs for SFT before RL

**3 axioms**

every final proof re-checked with the standard Lean CLI; relies only on the three standard axioms

**Compare Part 1: same verifier, but there Lean steered a frozen model's search. Here Lean is the training signal, and the search policy itself changes.**

Hubert, Mehta, Sartran et al. (Google DeepMind), "Olympiad-level formal mathematical reasoning with reinforcement learning," Nature 651, 607–613, Nov 12 2025 · Fig. 1 (core components), Fig. 2a (main RL loop)

# Test-Time RL

- **For a hard target problem, a variant generator writes a curriculum of related formal problems: simplifications, generalizations, special cases. “Millions of related problem variants at inference time.”**
- The same AlphaZero-style RL loop then trains on that bespoke curriculum, with Lean as the reward, before attempting the target.
- Cost: 2–3 days of TTRL per IMO problem; the paper reports scaling at 50 TPU-days per problem.
- Two complementary scaling axes (Fig. 4): more tree search buys >10 absolute points from 2 TPU-minutes to 12 TPU-hours; TTRL on top of 12 TPU-hours of search buys another +15 absolute points.
- Authors’ framing: “deep, problem-specific adaptation.” The model that solves the problem is not the model that started.

CONNECTION

## Why this is the interesting part for us

TTRL is the group’s self-improvement question in miniature: the model generates its own training problems around a target and learns from a verifier, at inference time.

It works only because the verifier is exact. A variant that is mis-generated is still a valid Lean problem; a variant that is unprovable just yields no reward. There is no way to reward-hack a type-checker.

It is also why the Part 1 agents call AlphaProof as a tool: what they are buying is a policy that has already internalized a lot of search.

Hubert, Mehta, Sartran et al. (Google DeepMind), “Olympiad-level formal mathematical reasoning with reinforcement learning,” Nature 651, 607–613, Nov 12 2025 · Fig. 2b (TTRL loop), Fig. 4 (scaling with inference compute)

# Results (IMO Silver)

**28 / 42**

IMO 2024 with AlphaGeometry 2: AlphaProof solved P1, P2, P6; AG2 solved P4. Silver equivalent.

**P6**

the hardest problem of the competition, solved by only 5 human contestants

**100%**

miniF2F-valid with TTRL; near-perfect on miniF2F-test

**30% @ 300**

solved by the final agent with just 300 search simulations; earlier agents could not match this with far more

THE KEY FINDING

## What “learned reasoning” looks like in the data

As RL training proceeded, the agent needed “substantially fewer tree search simulations to achieve a given solve rate.” The authors read this as the network “internalizing effective proof strategies”: search that used to happen at inference time now happens inside the policy.

Held-out benchmarks improved throughout training, so it is not memorization of the curriculum.

LIMITATIONS

## Caveats the authors state themselves

Multi-day inference per problem, far beyond human time limits.

Successes are “primarily within the domain of advanced high-school and undergraduate competition mathematics,” which is “not open-ended” and operates “within a known fixed library of mathematical concepts.”

Open: “theory building,” and “mathematical taste, interestingness or beauty.”

Hubert, Mehta, Sartran et al. (Google DeepMind), “Olympiad-level formal mathematical reasoning with reinforcement learning,” Nature 651, 607–613, Nov 12 2025 · Fig. 3 (training curves), Fig. 4 (inference scaling)

# What is Reasoning Here?

## Derivation

Producing a valid chain from premises to conclusion.

AlphaProof doesn't learn this; Lean supplies it. Every tactic application is checked, so validity is never the model's job.

This is what Tier A certifies and nothing more.

## Search

Choosing which of many valid next steps to take so the goal is reached within budget.

This is what AlphaProof learns: a policy that proposes good tactics, trained by −1 per step and a verified terminal reward, then sharpened per problem by TTRL. The evidence is search moving into the weights.

## Representation

Inventing the definition or reformulation that makes a problem tractable.

Not learned. The authors say so: the system operates "within a known fixed library of mathematical concepts," and theory building and taste are open.

This is what Tao's Jacobian post did, and what no verifier rewards.

**Claim for discussion: a good verifier drives reasoning in the sense of search, at training time and at test time. It does not drive representation, because nobody has a verifier for "this is a useful new idea."**

Hubert, Mehta, Sartran et al. (Google DeepMind), "Olympiad-level formal mathematical reasoning with reinforcement learning," Nature 651, 607–613, Nov 12 2025 · Tzachristas, Tzachristas & Sui, "Open Problems Solved by LLMs? A Survey of Verifiable Mathematical Discovery," ACL 2026 Big Picture workshop, pp. 10–21 · Tao, Jacobian digestion, Jul 21 2026

# Takeaways

1. The results are real. The interesting variable is the verifier, not the model.

2. Where the verifier is exact (Lean, a certificate), progress is fast, cheap, and Tier A or B by construction. Where it is a person, the dominant failure is not wrong proofs but rediscovery and solving the wrong statement.

3. In every paradigm the model solves a slightly different problem than the one asked. The paradigm is defined by whether anything catches that.